signature guarantees only that the ID token was issued by the correct OP but does not by itself guarantee that it was issued in response to this specific request.

The simplest defense against these attacks is to use the authorization code flow with PKCE as recommended for all OAuth2 flows. In this case the ID token is only issued by the OP from the token endpoint in response to a direct HTTPS request from the client. If you decide to use a hybrid flow to receive an ID token directly in the redirect back from the authorization endpoint, then OIDC includes several protections that can be employed to harden the flow:

- The client can include a random `nonce` parameter in the request and verify that the same nonce is included in the ID token that is received in response. This prevents replay attacks as the nonce in a replayed ID token will not match the fresh value sent in the new request. The nonce should be randomly generated and stored on the client just like the OAuth `state` parameter and the PKCE `code_challenge`. Note that the nonce parameter is unrelated to a nonce used in encryption as covered in chapter 6.
- The client can request that the ID token is encrypted using a public key supplied during registration or using AES encryption with a key derived from the client secret. This prevents sensitive personal information being exposed if the ID token is intercepted. Encryption alone does not prevent replay attacks, so an OIDC nonce should still be used in this case.
- The ID token can include `c_hash` and `at_hash` claims that contain cryptographic hashes of the authorization code